KherveBook: A Native Desktop Computational Notebook Combining Python, Markdown, LaTeX, Spreadsheets and Drawings

Gwilherm Kerhervé*
Department of Materials, Imperial College London, London SW7 2AZ, United Kingdom

10 July 2026 — Version 0.1.123

Abstract

KherveBook is an open-source, Jupyter-inspired computational notebook implemented as a genuine native desktop application: every cell is rendered with Qt widgets and Python runs in the application's own in-process kernel, with no browser and no embedded web runtime for core cells. A single scrolling document mixes six cell types — runnable Python, Markdown, full LaTeX documents, live multi-sheet spreadsheets two-way linked to Python, SVG drawings, and JavaScript/HTML. It ships approximately 180 built-in scientific examples spanning ten materials-characterisation techniques, an AI assistant that can read and edit cells, per-notebook Git version history, and lossless Jupyter `.ipynb` import/export. KherveBook is written in Python with PyQt5 and released under the GNU General Public License v3.0 at `https://github.com/gkerherve/KherveBook`.

1 Statement of need

Computational notebooks are the lingua franca of data-driven science, but the dominant implementations — Jupyter [1] and, more recently, Marimo [2] — run in a web browser on top of a client–server architecture. For a desktop scientist this brings friction: a server process to manage, a browser tab as the interface, limited native file-system integration, and cell types restricted to code and text. Experimental scientists in particular often want a spreadsheet next to their code, a quick sketch next to their data, and properly typeset LaTeX in the same document.

KherveBook explores a different point in the design space: the notebook as a native desktop application. There is no browser and no notebook server; cells are Qt widgets and code executes in an in-process kernel with `numpy`, `matplotlib`, `pandas`, `scipy`, `sympy` and `dask` preloaded. This makes it well suited to teaching laboratories, offline instrument PCs, and users who find the Jupyter deployment stack disproportionate to their needs.

2 Functionality

Six cell types. Code cells provide `In [n]:` counters, stdout/stderr capture, Jupyter-style echo of the trailing expression, inline or interactive matplotlib figures, line numbers and syntax highlighting. Markdown cells render formatted notes (double-click to edit). LaTeX cells typeset full documents via the `tectonic` engine [3], falling back to matplotlib mathtext when tectonic is not installed. Sheet cells embed a multi-sheet spreadsheet with `=` formulas (Python and Excel-style functions, A1 references and ranges). Drawing cells offer pen/line/rectangle/ellipse/text

*ORCID: 0000-0002-6449-1828, `g.kerherve@imperial.ac.uk`

tools, with hand-off to the sibling vector editor KherveScribe. JavaScript/HTML cells host D3, Plotly, canvas or three.js content in an embedded web view.

Sheet–Python two-way link. A spreadsheet cell publishes its grid to code cells as `sheet1`; code reads it, e.g. into a pandas DataFrame, and writes back with `ks("A1", value)`. Editing a number and re-running updates the plot; cells can also run continuously for live simulations.

Scientific example library. Around 180 built-in examples cover mathematics, physics, chemistry, biology, statistics, finance and signal processing, plus complete example sets for ARPES, XPS, XRD, EDX, EELS, TGA, BET, SIMS, NMR and FTIR. Each references the real community library for the technique (lmfit [4], HyperSpy/eXSpy [5], pyGAPS, nmrglue, pymatgen, spectrochempy, ...) and falls back to faithful synthetic data so every example runs offline.

Reproducibility and interoperability. Each `.kbook` document's folder is its own Git repository with auto-commit on save, a painted history graph, branch/diff/restore, and push/pull to GitHub or GitLab. Import/export covers Jupyter/Colab `.ipynb` (lossless round-trip), `.ksheet`/`.kdoc`/`.ktex`, `.xlsx`/`.xlsm`, and images or PDFs as SVG cells. An AI assistant (Claude, ChatGPT, Mistral, Ollama or a local model) reads every cell and can add or rewrite cells on request.

3 Comparison with existing tools

	Jupyter	Marimo	KherveBook
Interface	Browser	Browser	Native Qt desktop app
Engine	IPython kernel	Reactive runtime	Own in-process kernel
Cells beyond code/text	—	—	Sheet, SVG, LaTeX, JS
Built in	Ecosystem	Reactive `.py` files	AI, Git, ∼180 examples

KherveBook shares the cell-based notebook idea with Jupyter and Marimo but none of their code. Marimo-style reactive execution is on the roadmap.

4 Availability

Source code, a Windows installer, a portable build and documentation are available at `https://github.com/gkerherve/KherveBook` under the GPL-3.0 license. KherveBook is part of the Kherve family of open-source scientific tools, which includes KherveFitting [6], KherveTeX and KhervePDF.

References

[1] T. Kluyver et al., *Jupyter Notebooks — a publishing format for reproducible computational workflows*, in: Positioning and Power in Academic Publishing, IOS Press (2016) 87–90.

[2] A. Agrawal, M. Scolnick, *marimo: a reactive notebook for Python*, `https://marimo.io`.

[3] P. K. G. Williams et al., *Tectonic: a modernized, complete, self-contained TeX/LaTeX engine*, `https://tectonic-typesetting.github.io`.

[4] M. Newville et al., *LMFIT: Non-linear least-square minimization and curve-fitting for Python*, doi:10.5281/zenodo.11813.

[5] F. de la Peña et al., *HyperSpy: multi-dimensional data analysis toolbox*, doi:10.5281/zenodo.592838.

[6] G. Kerhervé, D. J. Payne, *KherveFitting: An Open Source Software for Fitting X-Ray Photoelectron Spectroscopy Data*, Surf. Interface Anal. (2026), doi:10.1002/sia.70032.